Parallel XGBoost for scaling/accelerating stock price prediction

G. Hains (U. Paris-Est) A. Wong (Langara college) Y. Khmelevsky (Okanagan college)

June 2023

- Why
- What
- How
- How hard

Parallel XGBoost for scaling/accelerating stock price prediction

Why:

1. we have high-precision stock-price prediction on a limited time-window and using re-training a XGBoost model before each prediction.

2. To be attractive as a service we need to train on several hundreds of company stocks while there are only a few dozen currently. History for each company is currently set at 60 days (can this parameter also grow?)

3. Training time appears to be explosive w.r.t. number of companies.

Parallel XGBoost for scaling/accelerating stock price prediction

What:

Parallel computing to accelerate XGBoost at a given problem size and/or scale-up i.e. maintain training time for increasingly bigger problem sizes.

Due to its more naturally parallel training algorithm than neural nets, XGBoost can leverage almost any type of parallel hardware.

Due to its popularity, parallel execution is already supported:

(a) For multicore GPUs: using the ntreads parameter.

XGBoostThreads.pdf

(b) For single- or multiple (multi-node?) GPUs: using CUDA.

XGBoostGPUsupport.pdf

Parallel XGBoost for scaling/accelerating stock price prediction

How:

Parallel computing to accelerate XGBoost at a given problem size and/or scale-up i.e. maintain training time for increasingly bigger problem sizes.

Multicore execution on one CPU node requires nothing else than a parameter.

Single-GPU execution requires installing CUDA and seting up the GPU.

Multi-node (multi-GPU or multi-CPU) require installing Spark, MPI, and of course this hardware is only available at our computing centers.

For the sake of work efficiency and also because of the natural hardware hierarchy it will be natural to test those modes in sequence (apparently without coding) and if needed later mix them (may require parallel coding which takes training and time).

Parallel XGBoost for scaling/accelerating stock price prediction

How hard:

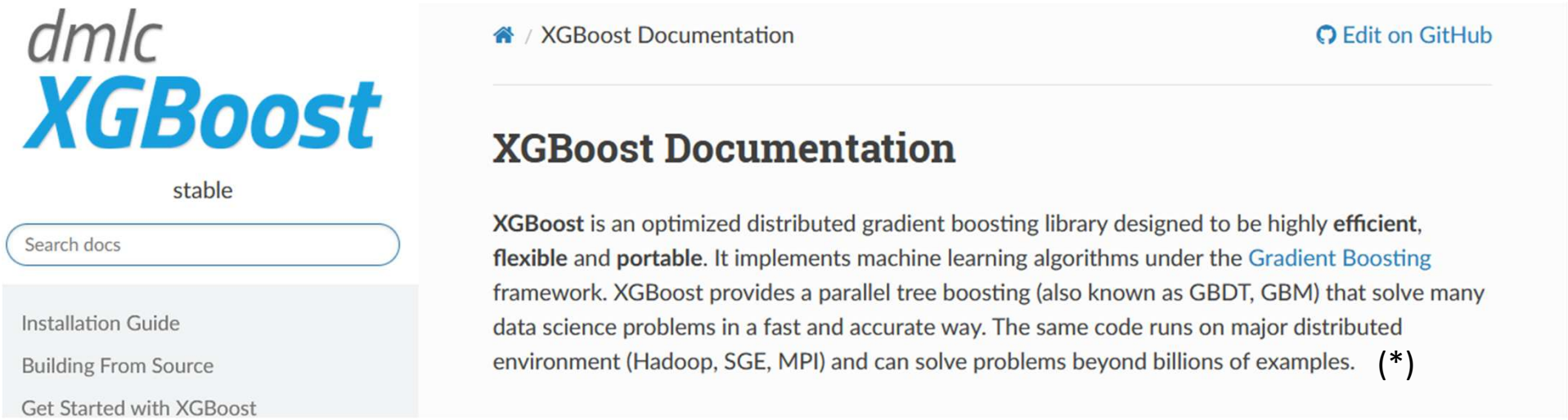
In theory a parallel training is not equivalent to a sequential training because it reorders certain dependencies. Let's hope XGBoost is robust enough to preserve our prediction quality when executed in parallel.

Speedup is in general less than the number of compute units: e.g. 5x faster on 16 cores. This is because of natural dependencies in the algorithms, weighted by the time needed for transfers. In this respect GPU is worse, but it has huge parallelism.

Multi-GPU is reported with the worse parallel efficiency because of internal GPU features + inter-node MPI communication that, costliest level of the memory hierarchy: 40% speedup only when going from 1GPU to 8GPU. But 1GPU is certainly much faster than 1CPU.

Parallel XGBoost for scaling/accelerating stock price prediction

In summary



The XGBoost parallel implementations are presented as *scale-up* solutions (*)
but that doesn't mean the are automatically *speed-up* tools.
MPI is known to utilize multi-node hardware to its best, but inter-node comm is costly.
Hadoop is fault-resistant and highly-scalable but known to be slow.
Spark utilises shared memory and scales less but should be much faster.